15 Advanced functional programming

This chapter covers

- Functional programming concepts
- Limits of functional programming in Java
- Kotlin advanced functional programming
- Clojure advanced functional programming

We have already met functional programming concepts earlier in the book, but in this chapter we want to draw together the threads and step it up. There is a lot of talk about *functional programming* in the industry, but it remains a rather ill-defined concept. The sole point that is agreed upon is that in a functional programming (FP) language, code is representable as a first-class data item, that is, that it should be possible to represent a piece of deferred computation as a value that can be assigned to a variable.

This definition is, of course, ludicrously broad—for all practical purposes, every mainstream language (with very few exceptions) in the last 30 years meets this definition. So, when different groups of programmers discuss FP, they are talking about different things. Each tribe has a different, tacit understanding of what other language properties are implicitly understood to *also* be included under the term "FP."

In other words—just as with OO—there is no fundamentally agreed-upon definition of what a "functional programming language" is. Alternatively, if everything is a FP language, then nothing is.

The well-grounded developer is well advised to visualize programming languages upon an axis (or, better yet, as a point in a multidimensional space of possible language characteristics). Languages are simply more or less functional than other languages—there is not some absolute scale that they are weighed against. Let's meet some of the concepts of the common toolbox of functional programming languages that go beyond the somewhat facile "code is data" notion.

15.1 Introduction to functional programming concepts

In what follows, we will frequently speak of *functions*, but neither the Java language nor the JVM has any such thing—all executable code must be expressed as a *method*, which is defined, linked, and loaded within a *class*. Other, non-JVM languages, however, have a different conception of executable code, so when we refer to a function in this chapter, it should be understood that we mean a piece of executable code that roughly corresponds to a Java method.

15.1.1 Pure functions

A *pure function* is a function that does not alter the state of any other entity. It is sometimes said to be *side-effect free*, which is intended to mean that the function behaves like an ideation of a mathematical function: it takes in arguments, does not affect them in any way, and returns a result that is dependent only on the values that have been passed.

Related to the concept of purity is the idea of *referential transparency*. This is somewhat unfortunately named—it has nothing to do with references as a Java programmer would understand them. Instead, it means that a function call can be replaced with the result of any previous call to the same function with the same arguments.

It is obvious that all pure functions are referentially transparent, but there may also exist functions that are not pure and yet are also referentially transparent. To allow a non-pure function to be considered in this way would require a formal proof based on code analysis. Purity is about code, but immutability is about data, and that's the next FP concept we will look at.

15.1.2 Immutability

Immutability means that after an object has been created, its state cannot be altered. The default in Java is for objects to be mutable. The keyword `final` is used in various ways in Java, but the one that concerns us here is to prevent modification of fields after creation. Other languages may favor immutability and indicate that preference in various ways—such as Rust, which requires programmers to explicitly make variables mutable with the `mut` modifier.

Immutability makes code easier to reason about: objects have a trivial state model, simply because they are constructed in the only state they will ever exist in. Among other benefits, this means that they can be copied and shared safely, even between threads.

NOTE We might ask whether any "almost immutable" approaches to data exist that still maintain some (or most) of the attractive properties of immutability. In fact, the Java `CompletableFuture` class that we have already met is one such example. We'll have more to say about this in the next chapter.

One consequence is that, because immutable objects cannot be altered, the only way that state change can be expressed in a system is by starting from an immutable value and constructing a completely new immutable value that is more or less the same but with some fields altered—possibly by the use of *withers* (aka `with*()` methods).

For example, the `java.time` API makes very extensive use of immutable data, and new instances can be created by the use of withers like this:

```
LocalDate ld = LocalDate.of(1984, Month.APRIL, 13);
LocalDate dec = ld.withMonth(12);
System.out.println(dec);
```

The immutable approach has consequences—specifically a potentially large impact on the memory subsystem, because the components of the old value have to be copied as part of the creation of the modified value. This means that in-place mutation is often much cheaper from a performance perspective.

5.1.3 Higher-order functions

Higher-order functions are actually a very simple concept, described by the following insight: if a function can be represented as a data item, then it should be able to be treated as though it was any other value.

We can define a *higher-order function* as a function value that does one or both of the following:

- Takes a function value as a parameter
- Returns a function value

Consider, for example, a static method that takes in a Java `String` and generates a function object from it, as shown here:

```
public static Function<String, String> makePrefixer(String prefix) {
  return s -> prefix +": "+ s;
}
```

This provides a straightforward way to make function objects. Let's now combine it with another static method, shown next, that this time accepts a function object as input:

```
public static String doubleApplier(String input,
                                   Function<String, String> f) {
  return f.apply(f.apply(input));
}
```

This provides us with the following simple example:

```
var f = makePrefixer("NaNa");                    ❶
System.out.println(doubleApplier("Batman", f));  ❷
```

❶ Creates a function object

❷ Passes the function object as a parameter to another method

However, this is not quite the whole story for Java, as we will see in the next section.

5.1.4 Recursion

A *recursive* function is one that calls itself on at least some of the code paths through the function. This leads to one of the oldest jokes in programming: "in order to understand recursion, one must first understand recursion."

However, to be more strictly accurate, we might write it as follows: in order to understand recursion, one must first understand

1. recursion, and
2. that in a physically realizable system, every chain of recursive calls must eventually terminate and return a value.

The second point is important: programming languages use call stacks to allow functions to call other functions, and this occupies space in memory. Recursion, therefore, has the problem that deep recursive calls may potentially use up too much memory and crash.

In terms of theoretical computer science, recursion is interesting and important for many different reasons. One of the most important is that recursion can be used as a basis to explore theories of computation and ideas such as *Turing completeness*, which is loosely

the idea that all nontrivial computation systems have the same theoretical capability to perform calculations.

5.1.5 Closures

A *closure* is usually defined as a lambda expression that "captures" some state from the surrounding context. However, for this definition to make sense, we need to explain the meaning of the capturing concept.

When we create a value and assign (or bind) it to a local variable, the variable will exist and can be used until some later point in the code. This later point may well be the end of the function or block where the variable was declared. The area of code where the variable exists and can be used is the *scope* of the variable.

When we create a function value, the local variables declared within the function body will still be in scope during the invocation of the function value, which will occur later than the point where the function value is declared. If, in the declaration of the function value, we mention a variable (or other state, such as a field) that is declared outside the scope of the function body, then the function value is said to have *closed over* the state, and the function value is known as a closure.

When the closure is later invoked, it has full access to the captured variables, even if the invocation happens in a different scope to that in which the capture was declared.

For example, in Java:

```
public static Function<Integer, Integer> closure() {
    var atomic = new AtomicInteger(0);
    return i -> atomic.addAndGet(i);
}
```

This static method is a higher-order function that returns a Java closure because it returns a lambda expression that references `atomic`, which was declared as a local variable inside the method, that is, in the scope where the lambda was itself declared. The closure returned from `closure()` can be called repeatedly, and it will aggregate state on each call.

5.1.6 Laziness

We briefly mentioned the concept of *laziness* in chapter 10. Essentially, *lazy evaluation* allows the computation of the value of an expression to be deferred until the value is actually required. By contrast, the immediate evaluation of an expression is known as *eager evaluation* (or *strict evaluation*).

The idea of laziness is simple: if you don't need to do work, don't do it! It sounds simple but has deep ramifications for how you write your programs and how they perform. A key part of this additional complexity is that your program needs to track what work has and hasn't been completed already.

Not every language supports lazy evaluation, and many programmers may have encountered only eager evaluation at this point in their journey—and that's completely OK.

For example, there is no general language-level support for laziness in Java, so it's difficult to give a clear example of the feature. We'll have to wait until we talk about Kotlin to make it concrete.

However, although laziness is not necessarily a natural concept for a Java programmer, laziness is an extremely useful and powerful technique in FP. In fact, for some FP languages, such as Haskell, lazy evaluation is the default.

5.1.7 Currying and partial application

Currying, unfortunately, has nothing to do with food. Instead, it is a programming technique named after Haskell Curry (who also gave his name to the Haskell programming language). To explain it, let's start with a concrete example.

Consider an eagerly-evaluated, pure function that takes two arguments. If we supply both arguments, we will get a value, and the function call can be replaced everywhere by the resulting value (this is referential transparency). But what happens if we supply not both but only one of the two arguments?

Intuitively, we can think of this as creating a new function, but one that needs only a single argument to calculate a result. This new function is called a *curried function* (or *partially-applied function*). Java does not have direct support for currying, so we will again defer making a concrete example until later in the chapter.

Looking farther afield, some programming languages support the notion of functions with multiple argument lists (or have syntax that allows the programmer to fake them).

In this case, another way to think about currying is as a transformation of functions. In mathematical notation, we are translating a multiple-argument function that is called as `f(a, b)` into one callable as `(g(a))(b)`, where `g(a)` is the partially applied function.

As should be obvious now, the different languages we've met so far have different levels of support for functional programming—for example, Clojure has very good support for many of the concepts that we have discussed in this section. Java, on the other hand, is a very different story, as we'll see in the next section.

15.2 Limitations of Java as a FP language

Let's start with the good news, such as it is: Java definitely clears the rather low bar of "represent code as data" via the types in `java.util.function` and also via the extensive introspective support the runtime provides (such as Reflection and Method Handles).

NOTE The use of inner classes to simulate function objects as a technique predates Java 8 and was present in libraries like Google Guava, so strictly speaking, Java's ability to represent code as data is not tied to that version.

Since version 8, the Java language goes somewhat further than the bare minimum with the introduction of streams and, with them, a heavily restricted domain of lazy operations. However, despite the arrival of streams, Java is not a naturally functional environment. Some of this is due to the history of the platform and what are—by now—decades-old design decisions.

NOTE It is worth remembering that Java is a 25-year-old imperative language that has been iterated upon extensively. Some of its APIs are amenable to FP, immutable data, and so on, and some are not. This is the reality of working in a language that has survived, and thrived, yet still remains backward compatible.

So, overall, Java is perhaps best described as a "slightly functional programming language." It has the basic features needed to support FP and provides developers with access to basic patterns like filter-map-reduce via the Streams API, but most advanced functional features are either incomplete or missing entirely. Let's take a detailed look.

15.2.1 Pure functions

As we saw in chapter 4, Java's bytecodes do many different sorts of things, including arithmetic, stack manipulation, flow control, and especially invocation and data storage and retrieval. For well-grounded developers who already understand JVM bytecode, this

means that we can express purity of methods by thinking about the effect of bytecodes. Specifically, a pure method in a JVM language is one that does the following:

- Does not modify object or static state (does not contain `putfield` or `putstatic`)
- Does not depend upon external mutable object or static state
- Does not call any non pure method

This is a pretty restrictive set of conditions and underlines the difficulty of using the JVM as a basis for pure functional programming.

There is also a question about the semantics—that is, the intent—of the different interfaces present in the JDK. For example, `Callable` (in `java.util.concurrent`) and `Supplier` (in `java.util.function`) both do basically the same thing: they perform some calculation and return a value, as shown here:

```
@FunctionalInterface
public interface Callable<V> {
   V call() throws Exception;
}

@FunctionalInterface
public interface Supplier<T> {
   T get();
}
```

They are both `@FunctionalInterface` and are both routinely used as the target type for a lambda. The signatures of the interfaces are the same, apart from different approaches to handling exceptions.

However, they can be seen as having different roles: a `Callable` implies potentially non-trivial amounts of work in the called code to create the value that will be returned. On the other hand, the name `Supplier` seems to imply less work—perhaps just returning a cached value.

5.2.2 Mutability

Java is a mutable language—mutability is baked into its design from the earliest days. Partly this is an accident of history—the machines of the late 1990s (from when Java hails) were very restricted (by modern standards) in terms of memory. An immutable data model would have greatly increased stress on the memory management subsystem, and caused much more frequent GC events, leading to far worse throughput.

Java's design, therefore, favors mutation over the creation of modified copies. So in-place mutation can be seen as a design choice caused by performance trade-offs from 25 years ago.

The situation, however, is even worse than that. Java refers to all composite data by reference, and the `final` keyword applies to the reference, *not* to the data. For instance, when applied to fields, the field can be assigned to only once.

This means that even if a object has all final fields, the composite state can still be mutable because the object can hold a `final` reference to another object that has some nonfinal fields. This leads to the problem of shallow immutability, as we discussed in chapter 5.

NOTE For C++ programmers: Java has no concept of `const`, although it does have it as an (unused) keyword.

For example, here is a slightly enhanced version of the immutable `Deposit` class that we met in chapter 5:

```
public final class Deposit implements Comparable<Deposit> {
   private final double amount;
   private final LocalDate date;
   private final Account payee;

   private Deposit(double amount, LocalDate date, Account payee) {
      this.amount = amount;
      this.date = date;
      this.payee = payee;
   }

   @Override
   public int compareTo(Deposit other) {
      return Comparator.nullsFirst(LocalDate::compareTo)
                  .compare(this.date, other.date);
   }

   // methods elided
}
```

The immutability of this class rests upon the assumption that `Account` and all of its transitive dependencies are also immutable. This means there are limits to what can be

done—fundamentally the data model of Java and the JVM is not naturally friendly to immutability.

In the bytecode, we can see that finality fields shows up as a piece of field metadata as follows:

```
$ javap -c -p out/production/resources/ch13/Deposit.class
Compiled from "Deposit.java"
public final class ch13.Deposit
    implements java.lang.Comparable<ch13.Deposit> {

    private final double amount;

    private final java.time.LocalDate date;

    private final ch13.Account payee;

  // ...
  }
```

Trying to use immutable approaches to state in Java is bailing out a leaking boat. Every single reference has to be checked for mutability, and if even one is missed, then the entire object graph is mutable.

Worse still, the JVM's reflection and other subsystems also provide ways to circumvent immutability, as shown next:

```
var account = new Account(100);
var deposit = Deposit.of(42.0, LocalDate.now(), account);
try {
    Field f = Deposit.class.getDeclaredField("amount");
    f.setAccessible(true);
    f.setDouble(deposit, 21.0);
    System.out.println("Value: "+ deposit.amount());
} catch (NoSuchFieldException e) {
    e.printStackTrace();
} catch (IllegalAccessException e) {
    e.printStackTrace();
}
```

Taken together, all of this means that neither Java nor the JVM is an environment that provides any particular support for programming with immutable data. Languages like

Clojure, which have stronger requirements, end up having to do a lot of the work in their language-specific runtime.

5.2.3 Higher-order functions

The concept of a higher-order function should not be surprising to a Java programmer. We have already seen an example of a static method, `makePrefixer()`, that takes in a prefix string and returns a function object. Let's rewrite the code and change the static factory into another function object like this:

```
Function<String, Function<String, String>> prefixer =
                                prefix -> s -> prefix +": "+ s;
```

This can be a little hard to read at first glance, so let's include some extra bits of syntax that we don't actually need, to make what's going on clearer:

```
Function<String, Function<String, String>> prefixer = prefix -> {
   return s -> prefix +": "+ s;
};
```

In this expanded view, we can see that `prefix` is the argument to the function and the returned value is a lambda (actually a Java closure) that implements `Function <String, String>`.

Notice the appearance of the function type `Function<String, Function<String, String>>` —it has two type parameters that define the input and output types. The second (output) type parameter is just another type—in this case, it is another function type. This is one way to recognize a higher-order function type in Java: a `Function` (or other functional type) that has `Function` as one of its type parameters.

Finally, we should point out that language syntax does matter—after all, function objects can be created as anonymous implementations, like this:

```
public class PrefixerOld
   implements Function<String, Function<String, String>> {

   @Override
   public Function<String, String> apply(String prefix) {
      return new Function<String, String>() {
```

```
        @Override
        public String apply(String s) {
            return prefix +": "+ s;
        }
      };
    }
}
```

This code would even have been legal as far back as Java 5, if the `Function` type had existed back then (and as far back as Java 1.1, if we remove the annotations and generics). But it's a total eyesore. It's very hard to see the structure, which is why many programmers think of functional programming as arriving only with Java 8.

5.2.4 Recursion

The `javac` compiler provides a straightforward translation of Java source code into bytecode. As we can see here, this applies to recursive calls:

```
public static long simpleFactorial(long n) {
    if (n <= 0) {
        return 1;
    } else {
        return n * simpleFactorial(n - 1);
    }
}
```

which compiles to the following bytecode:

```
public static long simpleFactorial(long);
  Code:
    0: lload_0
    1: lconst_0
    2: lcmp
    3: ifgt         8
    6: lconst_1
    7: lreturn
    8: lload_0
    9: lload_0
   10: lconst_1
   11: lsub
   12: invokestatic  #37                 // Method simpleFactorial:(J)J
```

```
      15: lmul
      16: lreturn
```

This, of course, has some major limitations. In this case, making a call like `simpleFactorial(100000)` will result in a `StackOverflowError` because of the `invokestatic` call at byte 12, which will cause an additional interpreter frame to be placed on the stack for each recursive call.

NOTE A *recursive* method is one that calls itself. A *tail-recursive* method is one where the self-call is the last thing that the method does.

Let's try to find a way to see whether the recursive call could be avoided. One approach is to rewrite the factorial code into a tail-recursive form, which in Java we can do most easily with a private helper method, as follows:

```
    public static long tailrecFactorial(long n) {
        if (n <= 0) {
            return 1;
        }
        return helpFact(n, 1);
    }

    private static long helpFact(long i, long j) {
        if (i == 0) {
            return j;
        }
        return helpFact(i - 1, i * j);
    }
```

The entry method, `tailrecFactorial()`, does not do any recursion; it merely sets up the tail-recursive call and hides the details of the more complex signature from the user. The bytecode for the method is basically trivial, but let's include it for completeness:

```
  public static long tailrecFactorial(long);
    Code:
      0: lload_0
      1: lconst_0
      2: lcmp
      3: ifgt          8
      6: lconst_1
      7: lreturn
      8: lload_0
```

```
 9: lconst_1
10: invokestatic  #49                 // Method helpFact:(JJ)J
13: lreturn
```

As you can see, there are no loops and only a single branching `if` at bytecode 3. The real action (and the recursion) happens in `helpFact()`. This is still compiled by `javac` into bytecode, which contains a recursive call, as we can see:

```
private static long helpFact(long, long);
  Code:
    0: lload_0
    1: lconst_0
    2: lcmp
    3: ifne         8
    6: lload_2                          (1)
    7: lreturn                          (2)
    8: lload_0
    9: lconst_1
   10: lsub
   11: lload_0
   12: lload_2
   13: lmul
   14: invokestatic  #49  // Method helpFact:(JJ)J    (3)
   17: lreturn
```

(1) Longs are 8 bytes, so they need two local variable slots each

(2) Returns from the i == 0 path

(3) Tail-recursive call

In this form, however, we can now see that there are two paths through this method. The simple `i == 0` path starts at bytecode 0, falls through the `if` condition at 3 and returns `j` at bytecode 7. The more general case is 0 to 3, then 8 to 14, which triggers a recursive call.

So, on the only path that has a method call on it, the call is recursive and always the last thing that happens before the `return`—that is, the call is in *tail position*. However, it *could* be compiled into the following bytecode instead, which avoids the call:

```
private static long helpFact(long, long);
  Code:
    0: lload_0
    1: lconst_0
    2: lcmp
    3: ifne       8
    6: lload_2          (1)
    7: lreturn          (2)
    8: lload_0
    9: lconst_1
   10: lsub
   11: lload_0
   12: lload_2
   13: lmul
   14: lstore_2         (3)
   15: lstore_0         (3)
   16: goto       0     (4)
```

(1) Longs are 8 bytes, so they need two local variable slots each

(2) Returns from the i == 0 path

(3) Resets the local variables

(4) Jumps to the top of the method

Now for the bad news: `javac` does not perform this operation automatically, despite it being possible. This is yet another example of how the compiler tries to translate Java source into bytecode as exactly as possible.

NOTE In the Resources project that accompanies this book is an example of how to use the ASM library to generate a class that implements the previous bytecode sequence because `javac` will not emit it from recursive code.

For completeness, we should say that, in practice, implementing a factorial function that handles longs with recursive calls rather than overwriting frames is not actually going to cause a problem, because the factorial increases so quickly that it will overflow the available space in a `long` well before any stack size limits are reached, as shown here:

```
$ java TailRecFactorial 20
2432902008176640000
```

```
$ jshell
jshell> 2432902008176640000L + 0.0
$1 ==> 2.43290200817664E18

jshell> Long.MAX_VALUE + 0.0
$2 ==> 9.223372036854776E18
```

So `factorial(21)` is already larger than the largest positive `long` that the JVM can express. However, although this specific trivial example is reasonably safe, it does not alter the difficult fact that all recursive algorithms in Java are potentially vulnerable to stack overflow.

This specific flaw is one of the Java language—and not of the JVM. Other languages on the JVM can, and do, handle this differently, for example, by use of an annotation or a keyword. We will see examples of this when we discuss how Kotlin and Clojure handle recursion later in the chapter.

5.2.5 Closures

As we've already seen, a closure is essentially a lambda expression that captures some visible state from the scope in which the lambda is declared, like this:

```
int i = 42;
Function<String, String> f = s -> s + i;
// i = 37;
System.out.println(f.apply("Hello "));
```

When this runs, it produces, as expected: `Hello 42`. However, if we uncomment the line that reassigns the value of `i`, then something different happens: the code stops compiling at all.

To understand why this happens, let's take a look at the bytecode that the code is compiled into. As we'll see in chapter 17, the bodies of lambda expressions in Java are turned into private static methods. In this case, the lambda body turns into this:

```
private static java.lang.String lambda$main$0(int, java.lang.String);
  Code:
    0: aload_1
    1: iload_0
    2: invokedynamic #32,  0 // InvokeDynamic #1:makeConcatWithConstants:
```

```
                    // (Ljava/lang/String;I)Ljava/lang/String;
    7: areturn
```

The clue is in the signature of `lambda$main$0()`. It takes *two* parameters, not one. The first parameter is the value of `i` that is passed in—which is 42 at the time that the closure is created (the second is the `String` parameter that the lambda takes when executed). Java closures contain copies of *values*, which are bit patterns (whether of primitives or object references) and not *variables*.

NOTE Java is strictly a *pass-by-value* language—there is no way in the core language to *pass-by-reference* or *pass-by-name*.

To see the effect of changes to captured state outside the scope of the closure body (or to effect things in other scopes), the captured state must be a mutable object, like this:

```
var i = new AtomicInteger(42);
Function<String, String> f = s -> s + i.get();
i.set(37);                                  (1)
// i = new AtomicInteger(37);               (2)
System.out.println(f.apply("Hello "));
```

(1) Reassigning a value to mutable object state works.

(2) This would fail to compile.

In fact, in earlier versions of Java, only variables that were explicitly marked as `final` could have their value captured by Java closures. However, from Java 8 onward, that restriction was changed to variables that are *effectively final*—variables that are used as though they were final, even if they don't actually have the keyword attached to their declaration.

This is actually symptomatic of a deeper problem. The JVM has a shared heap, method-private local variables, and a method-private evaluation stack, and that's it. As compared to other languages, neither the JVM nor the Java language has the concept of an *environment* or a symbol table, or the ability to pass a reference to an entry in one.

Non-Java languages on the JVM that do have those concepts are required to support them in their language runtime because the JVM does not provide any intrinsic support for them. Some programming language theorists, therefore, reach the conclusion that what Java provides are not actually true closures, because of the additional level of indi-

rection required. A Java programmer must mutate the state of an object value, rather than being able to change the captured variable directly.

5.2.6 Laziness

Java does not provide first-class support for lazy evaluation in the core language for ordinary values. However, one interesting place where we can see lazy evaluation in use is within the Java Streams API. Appendix B has a refresher on aspects of streams, should you require one.

NOTE Laziness does play a role in some parts of the JVM and its programming environment (e.g., aspects of class loading are lazy).

Calling `stream()` on a Java collection produces a `Stream` object, which is effectively a lazy representation of an ensemble of elements. Some streams can also be represented as a Java collection; however, streams are more general and not every stream can be represented as a collection.

Let's look again at a typical Java `filter()` and `map()` pipeline:

```
          stream()  filter()  map()    collect()
Collection -> Stream -> Stream -> Stream -> Collection
```

The `stream()` method returns a `Stream` object. The `map()` and `filter()` methods (like almost all of the operations on `Stream`) are lazy. At the other end of the pipeline, we have a `collect()` operation, which *materializes* the contents of the remaining `Stream` back into a `Collection`. This *terminal* method is eager, so the complete pipeline behaves like this:

```
          lazy     lazy     lazy     eager
Collection -> Stream -> Stream -> Stream -> Collection
```

Apart from the materialization back into the collection, the platform has complete control over how much of the stream to evaluate. This opens the door to a range of optimizations that are not available in purely eager approaches.

It can sometimes be helpful to think of the lazy, functional mode of Java streams as being analogous to hyperspace travel in a science-fiction movie. Calling `stream()` is the

equivalent of jumping from “normal space” into a hyperspace realm where the rules are different (functional and lazy, rather than OO and eager).

At the end of the operational pipeline, a terminal stream operation jumps us back from the lazy functional world into “normal space,” either by re-materializing the stream into a `Collection` (e.g., via `toList()`) or by aggregating the stream, via a `reduce()` or other operation.

Use of lazy evaluation does require more care from the programmer, but this burden largely falls upon library writers, such as the JDK developers. However, a Java developer should be aware of and respect the rules of some aspects of the lazy nature of streams. For example, implementations of some of the `java.util.function` interfaces (e.g., `Predicate`, `Function`) should not mutate internal state or cause side effects. Violating this assumption can cause major problems if developers write implementations or lambdas that do.

Another important aspect of streams is that the stream objects themselves (the instances of `Stream` that are seen as intermediate objects within a pipeline of stream calls) are single shot. Once they have been traversed, they should be considered invalid. In other words, developers should not attempt to store or reuse a stream object, because the results of doing so are almost certainly incorrect and attempts may throw an error.

NOTE Placing a stream object into a temporary variable is almost always a code smell, although doing so during development when debugging a complex generics issue with a stream is acceptable, provided the use of stream temporaries is removed when the code is completed.

One other aspect of the laziness of streams is the ability to model more general data than collections. For example, it is possible to construct an infinite stream by use of `Stream.generate()` combined with a generating function. Let’s take a look:

```
public class DaySupplier implements Supplier<LocalDate> {
    private LocalDate current = LocalDate.now().plusDays(1);

    @Override
    public LocalDate get() {
        var tmp = current;
        current = current.plusDays(1);
        return tmp;
    }
}
```

```
final var tomorrow = new DaySupplier();
Stream.generate(() -> tomorrow.get())
    .limit(10)
    .forEach(System.out::println);
```

This produces a stream of days that is infinite (or as large as is needed, if you prefer). This would be impossible to represent as a collection without running out of space, thus showing that streams are more general.

This example also shows that Java's restrictions, such as pass by value, restrict the design space somewhat. The `LocalDate` class is immutable, and so we are required to have a class containing a mutable field `current` and then to mutate `current` within the `get()` method to provide a stateful method that can generate a sequence of `LocalDate` objects.

In a language that supported pass by reference, the type `DaySupplier` would have been unnecessary, because `current` could have been a local variable declared in the same scope as `tomorrow`, which could then have been a lambda.

5.2.7 Currying and partial application

We already know that Java does not have any language-level support for currying, but we can take a quick look at how something could have been added. For example, here is the declaration for the `BiFunction` interface in `java.util.function`:

```
@FunctionalInterface
public interface BiFunction<T, U, R> {
   R apply(T t, U u);

   default <V> BiFunction<T, U, V> andThen(
                       Function<? super R, ? extends V> after) {

      Objects.requireNonNull(after);
      return (T t, U u) -> after.apply(apply(t, u));
   }
}
```

Notice how the default methods feature of interfaces is used to define `andThen()`—an additional method, beyond the standard `apply()` method for the `BiFunction`. This same technique could have been used to provide some support for currying, for example, by defining two new default methods as follows:

```
default Function<U, R> curryLeft(T t) {
    return u -> this.apply(t, u);
  }

  default Function<T, R> curryRight(U u) {
    return t -> this.apply(t, u);
  }
```

These define two ways to produce Java `Function` objects, that is, functions of one argument from our original `BiFunction`. Notice that they are implemented as closures. We're simply capturing the supplied value and storing it for later, when we actually apply the function. We could then use these additional default methods like this:

```
BiFunction<Integer, LocalDate, String> bif =
                          (i, d) -> "Count for "+ d + " = "+ i;

Function<LocalDate, String> withCount = bif.curryLeft(42);
Function<Integer, String> forToday = bif.curryRight(LocalDate.now());
```

However, the syntax is somewhat clunky: it requires two different methods for the two possible currys, and they must have different names due to type erasure. Even after all that, the resulting feature is arguably of only limited use, so this approach was never implemented, and, as discussed, Java does not support currying out of the box.

5.2.8 Java's type system and collections

To conclude our unfortunate tale about Java's less-than-great affinity to functional programming, let's talk about Java's type system and collections. The following three main issues with these parts of the Java language contribute to the somewhat poor fit to the functional programming style:

- Non-single-rooted type system
- `void`
- Design of the Java Collections

First of all, Java has a non-single-rooted type system (i.e., there is no common supertype of `Object` and `int`). This makes it impossible to write `List<int>` in Java and, as a result, leads to autoboxing and the attendant problems.

NOTE Lots of developers complain about the erasure of type parameters of generic types during compilation, but in reality, it is more often the non-single-rooted type system that really causes problems with generics in the collections.

Java has another problem, connected to the non-single-rooted type system: `void`. This keyword indicates that a method does not return a value (or, looked at another way, that the evaluation stack of the method is empty when the method returns). The keyword therefore carries the semantics that whatever the method does, it is acting purely by side effect—it's the opposite of "pure" in some sense.

The existence of `void` means that Java has both statements and expressions and that it is impossible to implement the design principle, "everything is an expression," which some functional programming traditions are very keen on.

NOTE In chapter 18, we will discuss Project Valhalla, which provides an opportunity for the Java language designers to potentially revisit the non-single-rooted nature of Java's type system (among other goals).

Another problem is related to the shape and nature of the Java Collections interfaces. They were added to the Java language with version 1.2 (aka Java 2), which was released in December 1998. They were not designed with functional programming in mind.

A major problem for doing FP with the Java Collections is that an assumption of mutability is built in everywhere. The Collections interfaces are large and explicitly contain methods like these from `List<E>`:

- `boolean add(E e)`
- `E remove(int index)`

These are mutation methods—the signature of them implies that the Collection object itself has been modified in place.

The corresponding methods on an immutable list would have signatures such as `List<E> add(E e)` that return a new, modified copy of the list. The case of `remove()` would be difficult to implement, because Java doesn't have the ability to return multiple values from a method.

NOTE The real problem is that `remove()` is incorrectly factored for FP, a very similar case to that of the `Iterator` that we discussed in section 10.4.

All of this therefore implies that *any* implementation of the Collections is implicitly expected to be mutable. There does exist the horrible hack of using `UnsupportedOperationException`, which we discussed in section 6.5.3, but this is not something that a well-grounded Java developer should use.

Other, non-Java languages separate out the concept of the collection type from mutability, for example, by representing them as different interfaces (or different *traits* in languages that support that concept). This enables implementations to specify whether or not they are mutable at type level, by choosing to implement—or not—the separate interfaces.

Lurking behind all of this is that one of Java's main virtues and most important design principles is backward compatibility. This makes it difficult, or impossible, to change some of these aspects to make the language more functional. For example, in the case of the Collections, rather than try to add additional, functional methods directly onto the Collections interfaces, a clean break was made and `Stream` was introduced, to act as a new container type that did not have the implicit semantics of the Collections.

Of course, just introducing a new container type and API does nothing to change the millions upon millions of lines of existing code that use the collections. Nor does it help in the slightest for the common case where an API is already expressed in terms of a collection type.

NOTE This problem is not unique to the stream/collection divide. For example, Java Reflection was introduced in Java 1.1 and predates the arrival of the Collections. As a result, the API is annoyingly difficult to use, because it relies upon arrays as element containers.

This section has shown some rather depressing facts about the state of support for functional programming in Java. The takeaway message is that simple functional patterns (such as filter-map-reduce) are available. These are very useful for all sorts of applications as well as generalizing well to concurrent (and even distributed) applications, but they are about the limit of what Java is able to do. Let's move on to look at our non-Java languages and see if the news is any better.

15.3 Kotlin FP

We've already demonstrated how modern Java handles some basic, common patterns in the functional programming paradigm. It probably comes as no surprise that Kotlin brings conciseness and a few additional ideas to the table for the functionally inclined.

This section will hit the high points, but take a look at *Functional Programming in Kotlin* by Marco Vermeulen, Rúnar Bjarnason, and Paul Chiusano (Manning, 2021, http://mng.bz/o2Wr) if you want to go even deeper.

5.3.1 Pure and higher-order functions

Back in section 9.2.4, we introduced Kotlin's functions. In Kotlin, functions are part of the type system, expressed with syntax like `(Int) -> Int`, where the contents of the parenthesized list are the argument types and to the right of the arrow is the return type.

By using this notation, we can easily write signatures for functions that accept other functions as arguments or return a function—that is, *higher-order functions*. Kotlin naturally encourages the use of such higher-order functions. Much of the API around working with collections such as `map` and `filter` are in fact built off these higher-order functions, just as we see in the Java (and Clojure) APIs that provide the equivalent language feature.

But higher-order functions aren't restricted to collections and streams. For instance, here's a classic functional programming function called `compose`. `compose` will return a function that calls each of the functions passed as arguments to it:

```
fun compose(callFirst: (Int) -> Int,
            callSecond: (Int) -> Int): (Int) -> Int {
  return { callSecond(callFirst(it)) }                  ❶
}

val c = compose({ it * 2 }, { it + 10 })                ❷
c(10)                                                   ❸
```

❶ compose returns a function, so callFirst and callSecond aren't called when this line executes.

❷ We pass two lambdas, using the it shorthand described in chapter 9 to avoid explicitly listing the single argument to the lambdas.

❸ We invoke and run the function returned by compose, which returns 30.

Kotlin provides a number of ways to get a handle to a function, depending on your needs. You can declare lambda expressions as shown earlier (and with many other fla-

vors and features discussed in chapter 9). Alternatively, we can refer to a named function via the `::` syntax as follows:

```
fun double(x: Int): Int {
  return x * 2
}

val c = compose(::double, { it + 10 })
c(10)                                  (1)
```

(1) Same result as our prior example

`::` knows more than just top-level functions. It can also refer to a function belonging to a specific object instance like this:

```
data class Multiply(var factor: Int) {
  fun apply(x: Int): Int = x * factor
}

val m = Multiply(2)
val c = compose(m::apply, { it + 10 })    (1)
c(10)
```

(1) References the apply method on the Multiply class bound specifically to our instance m.

Sadly, much like Java, Kotlin provides no built-in way of guaranteeing the purity of a given function. Although defining functions at the top level (outside of any class) and using `val` to ensure immutable data can take you a long way, they don't ensure the referential transparency of your functions.

5.3.2 Closures

An aspect of lambda expressions that may not be obvious on the surface is how they interact with the surrounding code. For instance, the following code works, even though `local` isn't declared within our lambda:

```
var local = 0
val lambda = { println(local) }
lambda()                        (1)
```

❶ Prints 0

This is referred to as *closure* (as in, the lambda *closes over* the values it can see). Importantly, and unlike Java, it is not just the value of the variables the lambda can access—under the covers, it actually keeps a reference to the variables themselves, as shown here:

```
var local = 0
val lambda = { println(local) }
lambda()                        ❶

local = 10
lambda()                        ❷
```

❶ Prints 0

❷ Prints 10, the updated value of local at the time lambda is invoked

This closure over variables remains even if the variables themselves would otherwise have gone out of scope. Here we return a lambda from a function, keeping a reference to a variable that normally would be inaccessible:

```
fun makeLambda(): () -> Unit {
  val inFunction = "I'm from makeLambda"    ❶
  return { println(inFunction) }
}

val lambda = makeLambda()
lambda()                                    ❷
```

❶ Because our lambda expression closes over inFunction, it is still available here—but only inside our lambda.

❷ inFunction would normally go out of scope when makeLambda is done.

NOTE Lambdas carrying references outside their typical scope can be a source for unexpected object leaks!

The location of a lambda expression's declaration determines what it may capture in its closure. For instance, if declared within a class, then the lambda can close over properties in the object as follows:

```
class ClosedForBusiness {
  private val amount = 100
  val check = { println(amount) }      ❶
}

fun getTheCheck(): () -> Unit {
  val closed = ClosedForBusiness()
  return closed.check                  ❷
}

val check = getTheCheck()
check()                                ❸
```

❶ A lambda saved into check closes over the private property amount.

❷ This function returns that lambda, which keeps a reference to amount. This keeps the instance closed alive when normally it wouldn't exist after the function completes.

❸ Prints 100 when called. When the check variable exits scope, the closed instance will also finally be eligible for garbage collection.

Closures with higher-order functions provide a rich basis for building new functions from old ones.

5.3.3 Currying and partial application

The currying story for Kotlin is very similar to that in Java. Let's look at an example:

```
fun add(x: Int, y: Int): Int {
  return x + y
}

fun partialAdd(x: Int): (Int) -> Int {
  return { add(x, it) }
}

val addOne = partialAdd(1)
println(addOne(1))
println(addOne(2))

val addTen = partialAdd(10)
println(addTen(1))
println(addTen(2))
```

This is really just a syntactic trick that works because the `()` operator desugars to a call to the `apply()` method. At bytecode level, this is really just the same as the Java example. We could imagine some helper syntax for automatically creating curries, perhaps something like

```
val addOne: (Int) -> Int = add(1, _)
println(addOne(10))
```

However, the core language does not directly support this. Various third-party libraries can provide similar, slightly more verbose abilities, often via an extension method.

5.3.4 Immutability

Section 15.1.2 framed immutability as a key technique for success in functional programming. If a pure function returns data that is meant to be identical for a given input, allowing an object to change after the fact breaks the guarantees that purity bought us.

A primary feature of Kotlin that aids in our quest for immutability is the `val` declaration. `val` ensures a property may be written only during object construction, much like Java's `final`. In fact, `val` is effectively the same as Java's `final var` combination, but is also applicable to properties and much less awkward to write.

Chapter 9 covers the many locations where Kotlin supports use of `val` / `var`, but to succeed in functional programming, it's recommended to embrace immutability and favor `val` over `var`. Kotlin's built-in support for properties also sweeps away the getter boilerplate required in Java, as shown next:

```
class Point(val x: Int, val y: Int)

val point = Point(10, 20)
println(point.x)
// point.x = 20  // Won't compile because x is immutable!
```

A major hangup with immutable objects, though, is the difficulty when you actually want to change something. To retain our immutability, we must create entirely new instances, but this can be tedious and error prone. In Java, this is often tackled with static factory methods, builder objects, or wither methods to cut down on the noise.

Kotlin's `data class` construct gives us a nice alternative to those approaches. In addition to the constructor and equality operations we covered in section 9.3.1, a data class also

gets a `copy` method. Pairing `copy` with Kotlin's named arguments, you can generate the new instance we want, writing out only the changes you actually want, as follows:

```
data class Point(val x: Int, val y: Int)

val point = Point(10, 20)
val offsetPoint = point.copy(x = 100)
```

`copy` does come with a couple important caveats. First, it is a shallow copy. If one of our fields is an object, we copy the reference to that object, not the full object itself. Like in Java, if any of the chain of objects allows mutation, then our guarantees are broken. For true immutability, all the objects involved need to play along, but the language will not enforce it for you.

Another point of caution is that `copy` is generated from the constructor of the class alone. If we bend our rules and put `var` fields elsewhere on our objects, `copy` doesn't know about these additional fields, and they get default values only in any copy, as shown here:

```
data class Point(val x: Int, val y: Int) {
  var shown: Boolean = false
}

val point = Point(10, 20)
point.shown = true

val newPoint = point.copy(x = 100)
println(newPoint.shown)                    (1)
```

(1) Outputs false, because the non constructor fields aren't touched by copy

But we'd never let a mutable field sneak into our nice immutable objects to begin with, right?

Controlling the mutability of our objects is an important first step, but most nontrivial code will involve collections of objects, not just individual instances. We saw in chapter 9 that Kotlin's functions for constructing collections (e.g., `listOf`, `mapOf`) return interfaces such as `kotlin.collections.List` and `kotlin.collections.Map`, which, unlike their counterparts in `java.util`, are read-only. Sadly, although this is a good start, it doesn't provide us the guarantees we want.

We can't trust the immutability of these objects because the mutable interfaces extend the read-only flavors. Anywhere you can pass a `List`, you can pass a `MutableList` as follows:

```
fun takesList(list: List<String>) {
  println(list.size)
}

val mutable = mutableListOf("Oh", "hi")
takesList(mutable)

mutable.add("there")
takesList(mutable)
```

`takesList` receives the same object in both invocations, but the result of the call is different. Our functionality purity is shattered!

NOTE The read-only helpers like `listOf` use underlying JDK collections and return objects that are read-only. For instance, `listOf` defaults to an array-backed list implementation that cannot be added to. It's just the mix of Kotlin's mutable interfaces with the standard read-only interfaces that spoil the party.

Implementing these collections via JDK classes also leaves some sharp edges if you cast between interfaces. Kotlin's aim for clean interoperation with Java Collections means the result from `listOf()` can be cast to both Kotlin's mutable interfaces and the classic `java.util.List<T>` where we can attempt to modify the collection! The following code compiles without a complaint but fails at runtime:

```
fun takesMutableList(muted: MutableList<Int>) {
  muted.add(4)
}

val list = listOf(1,2,3)
takesMutableList(list as MutableList<Int>)    ❶
```

❶ This call will throw a java.lang.UnsupportedOperationException.

This lack of real immutability becomes problematic especially when we're talking about concurrency. As we saw in chapter 6, making mutable collections safe between multiple threads takes quite an effort. If we had a collection instance that was truly immutable,

though, that could be freely distributed among different threads of execution, assured that everyone is getting an identical picture of the world.

Although Kotlin doesn't have them in the standard library, the `kotlinx.collections.immutable` library (see **http://mng.bz/nNjg**) provides a variety of immutable and *persistent* data structures. Frequently seen libraries such as Guava and Apache Commons also have many similar options.

What does it mean for a collection to be persistent? As we've discussed multiple times, immutability means that when you need to "change" an object, you instead create a new instance of it. For large collections, this could be really inefficient. Persistent collections lean on immutability to lower that cost of modification—they are built to safely share immutable parts of their internal storage. Though you still create a new object to effect any change, those new objects can be much smaller than a full copy, as shown next:

```
import kotlinx.collections.immutable.persistentListOf

val pers = persistentListOf(1, 2, 3)
val added = pers.add(4)
println(pers)                    (1)
println(added)                   (2)
```

(1) Prints [1, 2, 3]

(2) Prints [1, 2, 3, 4]

The core of the library implements two groups of interfaces typified by `ImmutableList` and `PersistentList`. Matching pairs exist for maps, sets, and general collections as well. `ImmutableList` extends `List`, but unlike its base interface, it guarantees any instance is immutable. `ImmutableList` can then be used in any locations where you're passing lists and want to enforce immutability. `PersistentList` builds off of `ImmutableList` to provide us with the "modify and return" methods.

The library also includes the following familiar extensions for converting other collections into persistent versions:

```
val mutable = mutableListOf(1,2,3)
val immutable = mutable.toImmutableList()
val persistent = mutable.toPersistentList()
```

At this point, you might be wondering why we don't change every `listOf` to `persistentListOf` "just in case." But these aren't the default implementation for a reason. Although persistent data structures lower the cost of copies, they still can't touch the speed of classic mutable data structures. Less copying *does not* equal no copying! How much does this cost?

As chapter 7 has hopefully convinced you, the only way to know is to measure in your own use cases. But if you need concurrent access to a collection across threads, it's worth comparing how these persistent structures perform versus more standard copying with synchronization. Now that we have Kotlin's toolkit for making data immutable in hand, let's look at a feature it brings to the world of recursive functions.

5.3.5 Tail recursion

In section 15.2.4, we examined recursive functions in Java. They have a major limitation because each successive recursive function call adds a stack frame, which eventually exhausts the available space. Kotlin has the same limitation with basic recursion, as we can see from the translation of our `simpleFactorial` function to Kotlin (note the use of a Kotlin `if` expression as the return value):

```
fun simpleFactorial(n: Long): Long {
  return if (n <= 0) {
    1
  } else {
    n * simpleFactorial(n - 1)
  }
}
```

This yields the following bytecode, which is equivalent to what `javac` emitted for our Java function:

```
public final long simpleFactorial(long);
    Code:
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifgt         10
       6: lconst_1
       7: goto         22
      10: lload_1
      11: aload_0
```

```
12: checkcast     #2                // class Factorial
15: lload_1
16: lconst_1
17: lsub
18: invokevirtual #19               // Method simpleFactorial:(J)J
21: lmul
22: lreturn
```

Apart from a little additional validation (byte 12) and the use of a `goto` instead of multiple `lreturn` instructions, this is fundamentally the same. The recursive call at byte 18 where we `invokevirtual` on `simpleFactorial` will eventually blow the stack, as shown here:

```
java.lang.StackOverflowError
    at Factorial.simpleFactorial(factorial.kts:32)
    at Factorial.simpleFactorial(factorial.kts:32)
    at Factorial.simpleFactorial(factorial.kts:32)
  ...
```

Although this problem is unavoidable in the general case, Kotlin can help us out if our function is tail-recursive. Remember that a tail-recursive function is one where the recursion is the last operation in the entire function. Earlier, we showed how at the bytecode level we could reset state and `goto` the top of the function instead of adding a stack frame. This transforms our recursive call into a loop, with no danger of overflowing the stack. Java didn't provide us any way to do this, but Kotlin does.

NOTE Any recursive function can be rewritten to be tail-recursive. It may require additional parameters, variables, and tricks to do the transformation, but it is always possible. Given that tail-recursive functions can be transformed into simple looping, this shows that any recursive function can also be implemented iteratively using only loop constructs.

Getting our factorial recursive call into the final position takes a little shuffling. As in Java, we split the function to retain the nice single-argument form for users and put the more complicated recursive function—which now needs multiple arguments—into a separate function like this:

```
fun tailrecFactorial(n: Long): Long {
  return if (n <= 0) {
    1
```

```
  } else {
    helpFact(n, 1)
  }
}

tailrec fun helpFact(i: Long, j: Long): Long {    ❶
  return if (i == 0L) {
    j
  } else {
    helpFact(i - 1, i * j)
  }
}
```

❶ Our helper is marked tailrec so Kotlin knows to look for tail recursion.

The entry function `tailrecFactorial` doesn't hide any surprises in the bytecode. It does our beginning range check and then hands off to our tail-recursive helper as follows:

```
public final long tailrecFactorial(long);
    Code:
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifgt         10
       6: lconst_1
       7: goto         19
      10: aload_0
      11: checkcast    #2                  // class Factorial
      14: lload_1
      15: lconst_1
      16: invokevirtual #10                 // Method helpFact:(JJ)J
      19: lreturn
```

Bytes 0–3 check for an early return implemented by bytes 6–7. If we need to make the recursive call, then it loads up the values needed to `invokevirtual` for `helpFact` at byte 16.

The important difference the `tailrec` keyword introduces shows up in the bytecode for `helpFact`, shown here:

```
public final long helpFact(long, long);
    Code:
```

```
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifne          10
       6: lload_3
       7: goto          26
      10: aload_0
      11: checkcast     #2                  // class Factorial
      14: pop
      15: lload_1
      16: lconst_1
      17: lsub
      18: lload_1
      19: lload_3
      20: lmul
      21: lstore_3
      22: lstore_1
      23: goto          0
      26: lreturn
```

The majority of this method is doing the logical checks and arithmetic of our factorial, but the key is at byte 23. Instead of doing an `invokevirtual` for `helpFact` to recurse, instead we simply `goto 0` and start the function again. With no invoke instructions present, we have no danger of stack overflows, which is great news. Who says `goto` is always hazardous?

Tail recursion is an elegant solution when your function can be rewritten in the proper form. But what if you ask for it on a function that isn't tail-recursive, as shown next:

```
tailrec fun simpleFactorial(n: Long): Long {    ❶
  return if (n <= 0) {
    1
  } else {
    n * simpleFactorial(n - 1)
  }
}
```

❶ Inappropriately asking for tailrec when our final call isn't ourselves—in this case, the final operation is the * on the result of the recursive call.

Kotlin spots the problem and warns us that it can't transform the bytecode to take advantage of tail recursion, pointing us straight to our incorrect, not-at-the-end call like

so:

```
factorial.kts:28:1: warning: a function is marked as tail-recursive
                    but no tail calls are found

tailrec fun simpleFactorial(n: Long): Long {
^
factorial.kts:32:9: warning: recursive call is not a tail call
  n * simpleFactorial(n - 1)
      ^
```

Issuing a warning is not an especially strong behavior for the compiler because it's possible that after a tail-recursive implementation has been created, it can be subsequently subtly modified to *not* be tail-recursive. Unless the build process flags the warning, this code can escape into production and cause a `StackOverflowError` at runtime. Arguably, it would be better if declaring a non-tail-recursive function as `tailrec` caused a compilation error, as is done in some other languages (e.g., Scala).

5.3.6 Lazy evaluation

As we mentioned earlier in the chapter, many functional languages (such as Haskell) rely heavily on *lazy evaluation*. As a language on the JVM, Kotlin doesn't center lazy evaluation in its core execution model. But it does bring first-class support for laziness where you want it via the `Lazy<T>` interface. This provides a standard structure for when you want to delay—or potentially skip entirely—a bit of processing.

Typically you don't implement `Lazy<T>` yourself, but instead use the `lazy()` function to construct instances. In the simplest form, `lazy()` takes a lambda, the return type of which determines the type `T` of the returned interface. The lambda is not executed until `value` is explicitly requested. We can also check whether we've calculated the value already as follows:

```
val lazing = lazy {
  42
}

println("init? ${lazing.isInitialized()}")  ❶
println("value = ${lazing.value}")          ❷
println("init? ${lazing.isInitialized()}")  ❸
```

❶ Checks whether we're initialized; will report false

❷ Access to value will force our lambda to execute and save the result.

❸ Checks whether we're initialized; will report true

Our desire to put off unneeded computation may overlap with the need to execute across multiple threads. When that happens, `lazy()` takes a `LazyThreadSafetyMode` enumeration to help control how that happens. The enum values are `SYNCHRONIZED` (the default for `lazy()`), `PUBLICATION`, and `NONE`, described here:

- `SYNCHRONIZED` uses the `Lazy<T>` instance itself to synchronize execution of the initializing lambda.
- `PUBLICATION` instead allows multiple concurrent executions of the initialization lambda but saves only the first value seen.
- `NONE` skips synchronization, with undefined behavior if accessed concurrently.

NOTE `LazyThreadSafetyMode.NONE` should be used only if you've 1) measured that synchronization in your lazy instances is an actual performance problem and 2) can somehow guarantee you'll never access the object from multiple threads. The other options, `SYNCHRONIZED` and `PUBLICATION`, can be chosen between based on whether your use case is sensitive to the initialization lambda running multiple times concurrently.

The `Lazy<T>` interface is designed to work with another advanced Kotlin feature called *delegated properties*. When defining a property on a class, instead of providing the value or a custom getter/setter, you can instead provide an object with the `by` keyword. That object must have an implementation of `getValue()` and (for `var` properties) `setValue()`. `Lazy<T>` fits this specification, as shown next, so we can easily put off initializing properties in our classes without repeating boilerplate or veering away from natural Kotlin syntax:

```
class Procrastinator {
  val theWork: String by lazy {
    println("Ok, I'm doing it...")   ❶
    "It's done finally"
  }
}

val me = Procrastinator()

println(me.theWork)                  ❷
```

```
println(me.theWork)          ❸
println(me.theWork)          ❸
```

❶ Diagnostic message to make it easier to prove everything works

❷ The first call to theWork will run the lambda and print the working message.

❸ Further calls to theWork, as seen on the two concluding lines, will just return the same, already-calculated value.

Much like immutability, laziness is excellent for our own objects but leaves us wondering about collections and iteration. Next we'll look at how Kotlin allows us to better control the flow of execution when streaming through collections with the `Sequence<T>` interface.

5.3.7 Sequences

Although Kotlin's collection functions are frequently convenient, they assume we will eagerly apply the function to the entire collection. An intermediate collection is created for each step in our chain of functions, as shown next—potentially a waste if we don't actually need the whole result:

```
val iter = listOf(1, 2, 3)
val result = iter
    .map { println("1@ $it"); it.toString() }    ❶
    .map { println("2@ $it"); it + it }          ❷
    .map { println("3@ $it"); it.toInt() }       ❸
```

❶ Generates an intermediate collection with ["1", "2", "3"]

❷ Generates an intermediate collection with ["11", "22", "33"]

❸ Generates our final result of [11, 22, 33]

If we follow the execution via figure 15.1, we can see each step of our chain of `map` calls happens on the full list before the next `map` runs.

```
val iter = listOf(1, 2, 3)
val result = iter
    .map { it.toString() }
    .map { it + it }
    .map { it.toInt() }
```

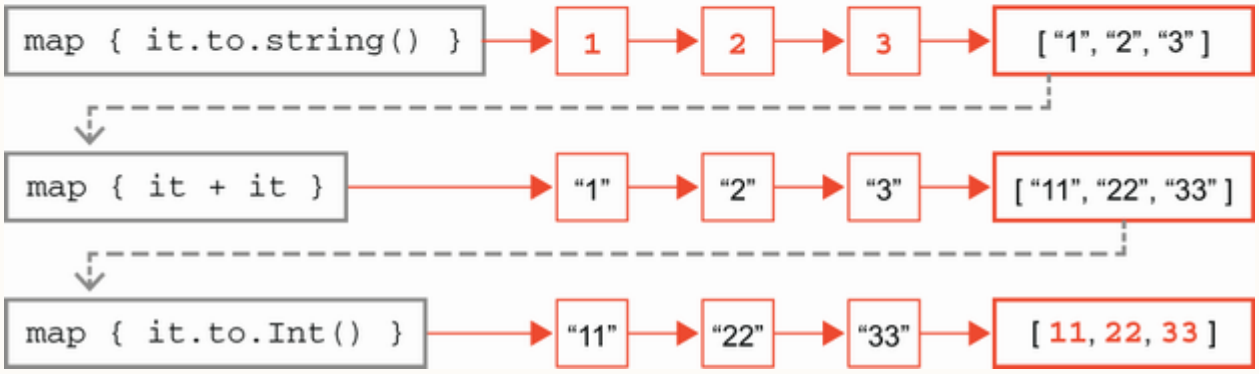


Figure 15.1 Standard iteration through collections

This results in the following output:

```
1@ 1
1@ 2
1@ 3
2@ 1
2@ 2
2@ 3
3@ 11
3@ 22
3@ 33
```

Beyond the possibility for wasted resources, there are also use cases where our inputs are potentially infinite. What if we wanted to continue this mapping across as many numbers as we can until the user tells us to stop? We can't create the list ahead of time and process the full thing step by step.

To handle this, Kotlin has *sequences*. At the core of sequences is the interface `Sequence<T>`, which looks similar to `Iterable<T>` but under the covers gives us a whole new set of capabilities.

We can create a new sequence using the `sequenceOf()` function and then start applying functions just like the collections we're used to. In the following example, we've turned our list into a sequence and kept the diagnostic prints so we can see what's going on:

```
val seq = sequenceOf(1, 2, 3)
val result = seq
    .map { println("1@ $it"); it.toString() }
    .map { println("2@ $it"); it + it }          (1)
    .map { println("3@ $it"); it.toInt() }
```

(1) Note that + here is string concatenation, not numeric addition.

When we run this short program we'll find that nothing prints. The primary feature of sequences is that they are lazy in their evaluation. Nothing in this program actually requires the return of our `map` calls, so Kotlin just doesn't run them! If we turn the sequence into a list using the following code, though, the program will be forced to evaluate everything and we can see how the control flow of the sequence runs:

```
val seq = sequenceOf(1, 2, 3)
val result = seq
    .map { println("1@ $it"); it.toString() }
    .map { println("2@ $it"); it + it }
    .map { println("3@ $it"); it.toInt() }
    .toList()
```

We can see in figure 15.2 that each element of the sequence goes through the `map` chain individually, first `1`, then `2`, and so on, before the next element is processed.

```
val seq = sequenceOf(1, 2, 3)
val result = seq
    .map { it.toString() }
    .map { it + it }
    .map { it.toInt() }
    .toList()
```

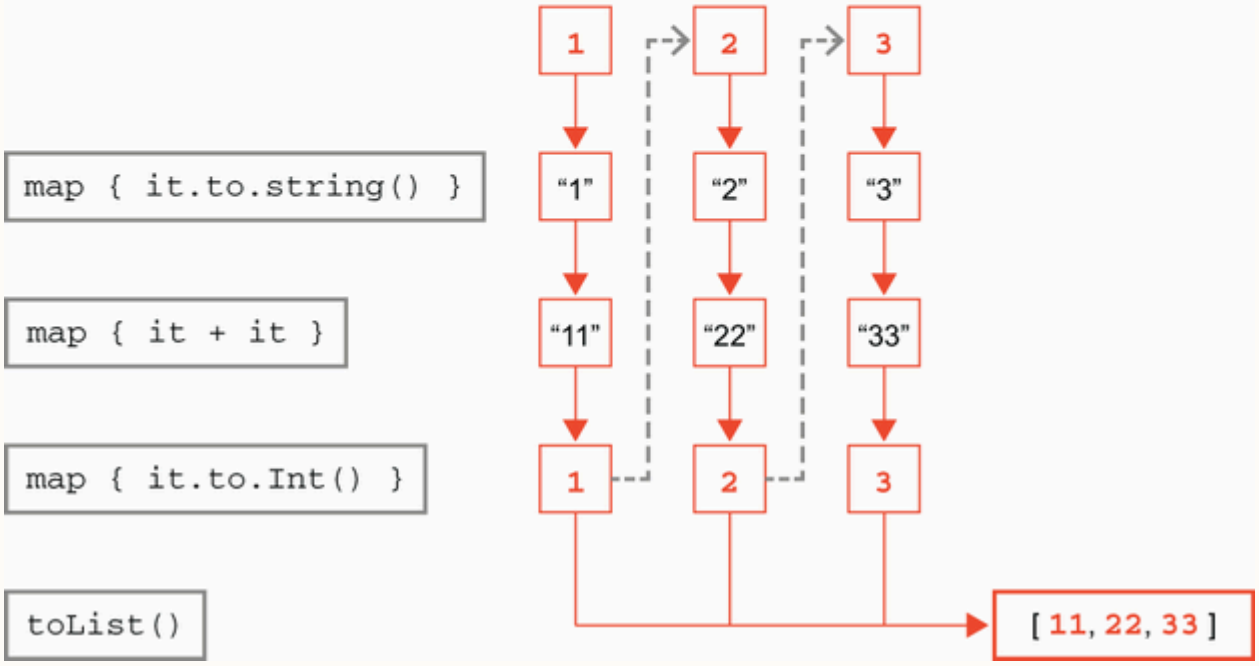


Figure 15.2 Sequence execution

This will have the following output:

```
1@ 1
2@ 1
3@ 11
1@ 2
2@ 2
3@ 22
1@ 3
2@ 3
3@ 33
```

This is interesting, but on relatively small, static lists, probably not that compelling—proper measurement is deserved, but the bookkeeping that sequences require may well drown out the wins for not allocating intermediate collections. The power of sequences becomes clearer with the alternative ways of creating sequences.

Our first stop is the `asSequence()` function. This will, unsurprisingly, turn things that are iterable into a sequence. The function works with more than just the lists and collections you might expect, though, and may be called on *ranges*.

We met Kotlin's numeric ranges back in section 9.2.6 where they were used to check inclusion with `when` expressions. But ranges may be iterated across, too. We can combine this with `asSequence()` to create long numeric lists without bothersome typing or over allocating, as shown next:

```
(0..1000000000)        ❶
  .asSequence()
  .map { it * 2 }
```

❶ Range tracks only its begin and end, so we don't create a billion elements.

But what if even the large-but-still-bounded nature of ranges feels too restrictive? `generateSequence()` from `kotlin.sequences` in the standard library has you covered. This function creates a new general sequence object with an optional starting value. Each time it needs the next element, it runs the provided lambda, passing in the prior value, like this:

```
generateSequence(0) { it + 2 }    ❶
```

❶ An infinite sequence of even numbers

An infinite `Iterable<T>` would be impossible to chain methods with because the first call on it would never return. `Sequence<T>` will just get what it needs and leave the rest for later. This pairs nicely with the `take()` function, where we can ask for a specific number of elements to be retrieved as a new, bounded sequence like so:

```
generateSequence(0) { it + 2 }
   .take(3)                ❶
   .forEach { println(it) }    ❷
```

❶ Creates a sequence with the first three elements in it

❷ Forces evaluation through the sequence and prints what we received

`forEach()` on sequences is what's referred to as *terminal*, because it ends the sequence's laziness and evaluates everything it has. We've seen another terminal already with `toList()`, which necessarily steps through every element to construct a list.

Kotlin offers another option for creating a sequence if, for some reason, it's difficult to work from just the prior element in the sequence. The pairing of `sequence()` with `yield()` lets us construct completely arbitrary sequences as follows:

```
val yielded = sequence {
  yield(1)
  yield(100)
  yieldAll(listOf(42, 3))    ❶
}
```

❶ yieldAll() takes an iterable of the same type we're yielding and will yield each element in turn when asked.

As we'd expect with sequences, the lambda is lazily executed to determine the next element. What's unique here, though, is that for each call for the next element, the lambda runs only until the next `yield` and then pauses. A subsequent request for another element will resume the lambda where it had paused and run until the next `yield` again, as shown in figure 15.3.



Figure 15.3 Timeline view of yield execution

`yield` uses a Kotlin feature known as *suspend* functions. As the name suggests, these are functions where Kotlin recognizes points in code execution that can stop and resume. In this case, Kotlin sees that each time we `yield` a value, execution of our `sequence` lambda should pause until the next value is requested. Though our code looks like a simple lambda, the Kotlin compiler is actually doing a lot of additional work for us behind the scenes.

Suspend functions are deeply related to Kotlin's alternative concurrency model, coroutines, which we introduced in section 9.4 and will discuss in more detail in chapter 16. It's an interesting point to note, though, that a feature that we often consider in light of concurrency also unlocks unique ways of functional programming as well.

15.4 Clojure FP

We met the basics of functional programming in Clojure in chapter 10 with forms like `(map)`. We also had early introductions to concepts like immutability and higher-order functions because those ideas and capabilities are very close to the core of the Clojure programming model.

So, in this section, rather than introducing the Clojure take on the features we discussed for Java and Kotlin, we will go beyond those foundations and show how some of Clojure's more advanced functional features work, starting with a note on list comprehensions.

15.4.1 Comprehensions

An important idea in functional programming is the concept of a *comprehension*, where the word means a "complete description" of a set or other data structure. This concept is derived from mathematics, where we often see set-theoretic notation used to describe sets, like this:

$$\{ x \in \mathbb{N} : (x / 2) \in \mathbb{N} \}$$

In this notation, $\in$ means "is a member of," $\mathbb{N}$ is the infinite set of all *natural numbers* (or counting numbers), which are the numbers we would use to count objects (so 1, 2, 3, and so on), and the `:` defines a condition or a *restriction*.

So this comprehension is describing a set of counting numbers that have a special property: every number in the set, when divided by two, yields a number that is also a counting number. Of course, we already know this set by another name—it is the set of even numbers.

The key point is that we did not specify the set of even numbers by listing out the elements (that would be impossible—there are an infinite number of them). Instead, we defined "even numbers" by starting from the natural numbers and specifying an additional condition that had to hold for each element to be included in the new set.

If that sounds a bit like the use of functional techniques, such as `filter`, then it should—they are very closely related concepts. However, functional languages often offer both comprehensions and `filter-map` because each approach turns out to be conceptually easier to use in different circumstances.

Clojure implements *list comprehensions* using the `(for)` form to return either a list (or an iterator, in some cases). This is why, when we met Clojure loops in chapter 10, we didn't introduce `(for)`—it isn't really a loop. Let's see it in action:

```
user=> (for [x [1 2 3]] (* x x))
(1 4 9)
```

The `(for)` form takes two arguments: an argument vector, and a form that will represent the values to *yield* as part of the overall list that `(for)` will return.

The argument vector contains a pair (or multiple pairs) of elements: a temporary that will be used in the definition of the yielded values and a seq to provide the inputs. We can think of the temporary as being used to bind each of the values, in turn. This could, of course, easily be written as a map like this:

```
user=> (map #(* % %) [1 2 3])
(1 4 9)
```

So, where would we want to use `(for)`? It comes into its own when we have more complex structures to build up, for example:

```
(for [num [1 2 3]
      ch [:a :b :c]]
  (str num ch))
("1:a" "1:b" "1:c" "2:a" "2:b" "2:c" "3:a" "3:b" "3:c")
```

We could also do this with a map, but the construction would potentially be more complex and cumbersome, whereas with `(for)` it is clear and straightforward. To get the effect of a filter, we can also use an additional qualifier on the `(for)` that can act as a restriction, as follows:

```
user=> (for [x (range 8) :when (even? x)] x)
(0 2 4 6)
```

Let's move on to look at how Clojure implements laziness, especially as applied to sequences.

5.4.2 Lazy sequences

In Clojure, laziness is mostly commonly seen when working with sequences rather than lazy evaluation of a single value. For sequences, laziness means that instead of having a complete list of every value that's in a sequence, values can instead be obtained when they're required (such as by calling a function to generate them on demand).

In the Java Collections, such an idea would require something like a custom implementation of `List`, and there would be no convenient way to write it without large amounts of boilerplate code. Using an implementation of `ISeq`, on the other hand, would allow us to write something like this:

```
public class SquareSeq implements ISeq {
    private final int current;

    private SquareSeq(int current) {
        this.current = current;
    }

    public static SquareSeq of(int start) {
        if (start < 0) {
            return new SquareSeq(-start);
        }
        return new SquareSeq(start);
    }

    @Override
    public Object first() {
        return Integer.valueOf(current * current);
    }

    @Override
    public ISeq rest() {
        return new SquareSeq(current + 1);
    }
}
```

There is no storage of values, and instead, each new element of the sequence is generated on demand. This allows us to model infinite sequences. Or consider this example:

```
public class IntGeneratorSeq implements ISeq {
  private final int current;
  private final Function<Integer, Integer> generator;
```

```
  private IntGeneratorSeq(int seed,
                          Function<Integer, Integer> generator) {
    this.current = seed;
    this.generator = generator;
  }

  public static IntGeneratorSeq of(int seed,
                                   Function<Integer, Integer> generator) {
    return new IntGeneratorSeq(seed, generator);
  }

  @Override
  public Object first() {
    return generator.apply(current);
  }

  @Override
  public ISeq rest() {
    return new IntGeneratorSeq(generator.apply(current), generator);
  }
}
```

This code sample uses the result of applying the function to provide the seed of the next sequence. This is fine, provided that the generator function is pure, but, of course, nothing guarantees that in Java. Let's move on and take a look at some powerful Clojure macros designed to help you create lazy seqs with only a small amount of effort.

Consider how you could represent a lazy, potentially infinite sequence. One obvious choice would be to use a function to generate items in the sequence. The function should do two things:

- Return the next item in a sequence
- Take a fixed, finite number of arguments

Mathematicians would say that such a function defines a *recurrence relation*, and the theory of such relations immediately suggests that recursion is an appropriate way to proceed.

Imagine you have a machine in which stack space and other constraints aren't present, and suppose that you can set up two threads of execution: one will prepare the infinite sequence, and the other will use it. Then you could use recursion to define the lazy seq in the generation thread with something like the following snippet of pseudocode:

```
(defn infinite-seq <vec-args>
 (let [new-val (seq-fn <vec-args>)]
  (cons new-val (infinite-seq <new-vec-args>))    ❶
 ))
```

❶ This doesn't actually work, because the recursive call to (infinite-seq) blows up the stack.

The solution is to add a construct that tells Clojure to optimize the recursion away and only proceed as needed: the `(lazy-seq)` macro. Let's look at a quick example in the next listing that defines the lazy sequence `k`, `k+1`, `k+2`, ... for some number k.

Listing 15.1 Lazy sequence example

```
(defn next-big-n [n] (let [new-val (+ 1 n)]
 (lazy-seq                          ❶
  (cons new-val (next-big-n new-val))          ❷
 )))

(defn natural-k [k]
 (concat [k] (next-big-n k)))                  ❸

1:57 user=> (take 10 (natural-k 3))
(3 4 5 6 7 8 9 10 11 12)
```

❶ lazy-seq marker

❷ Infinite recursion

❸ concat constrains recursion.

The key points are the form `(lazy-seq)`, which marks a point where an infinite recursion could occur, and the `(concat)` form, which handles it safely. You can then use the `(take)` form to pull the required number of elements from the lazy sequence. Lazy sequences are an extremely powerful feature, and with practice, you'll find them a very useful tool in your Clojure arsenal.

5.4.3 Currying in Clojure

Currying functions in Clojure has an additional complexity compared to other languages. This is caused by the fact that many Clojure forms are variadic, as we discussed in chap-

ter 10.

Variadic functions are a complication because they raise questions like: "Does the user mean to curry the two-argument form or evaluate the one-argument form?"—especially because Clojure uses eager evaluation of functions.

The solution is the `(partial)` form, which, by the way, is a genuine Clojure function, not a macro. Let's see it in action:

```
user=> (filter (partial = :b) [:a :b :c :b :d])
(:b :b)
```

The function `=` takes 1 or more arguments, and so `(= :b)` would eagerly evaluate to `true`, but the use of `(partial)` turns it into a curried function. Its use in a `(filter)` call causes it to be recognized as a function of one argument (effectively, the one-argument overload), and then it is used to test each element in the following vector.

NOTE `(partial)` by itself will curry only the first parameter of a form. If we wanted to curry some other parameter, we would need to combine `(partial)` with another form—one that permuted the argument list before function application.

Clojure is by far the most functional of the three languages we have looked at, and if the treatment here has whetted your appetite, you have a great deal more that you can explore. What we have covered so far is only the beginning, but it does help to demonstrate that the JVM itself can be a good home for functional programming—it's more the Java language that encumbers programming in a functional style.

In this chapter, we have delved far deeper into functional programming than just the traditional filter-map-reduce paradigm of Java Streams. We have largely done so by moving outside of Java to other JVM languages.

It is, of course, possible to go even further than this. Two major schools of functional programming exist: the dynamically typed school, as represented by Clojure (and languages like Erlang outside of the JVM), and the statically typed school, which includes Kotlin but is perhaps better represented by Scala (and Haskell for non-JVM languages).

However, one of Java's major design virtues—backward compatibility—can also be seen as a potential weakness. Java code that was compiled for version 1.0 (more than 25 years ago) will still run without modification on modern JVMs. However, this remarkable achievement does not come without a price tag. Java's APIs and even the design of the

bytecode and JVM have to live with design decisions that are difficult or impossible to change now and that are not especially friendly to FP. This is one major reason developers who want to use functional styles frequently find themselves moving to non-Java JVM languages.

Summary

- Filter-map-reduce is the starting point, not the end point, of functional programming.
- Java is not a language that is especially suited to a functional style, because it lacks built-in features like lazy evaluation, currying, and tail-recursion optimization.
- Other languages on the JVM can do a better job of supporting FP, with features such as Kotlin's `Lazy<T>` and Clojure's lazy sequences.
- Issues still exist at the JVM level, such as default mutability of data, that changing the choice of programming language cannot fundamentally fix.

Previous chapter
‹ Part 5. Java frontiers

Next chapter
16 Advanced concurrent programming ›